

Approvals as Memoized Decisions: Validity under Mutable World State in an Agent Runtime

Anonymous Author(s)

Abstract

An agent runtime that reuses a person’s approval must decide whether the world that justified it still holds. We measure one runtime across 272 arms and find that a recorded approval authorises an effect after its justifying world has changed — across a reload of its file-backed journal, and in the arm where the runtime looks, sees the change, and proceeds — while **every gate on the path returns a locally correct answer**. We argue that a human approval is a *memoized decision* whose validity key is the state the decision was about, and that this runtime memoizes it with a key that omits that state. The primitive is not new; the contribution is the identification and its measurement. From it we derive a property a runtime can be checked against: record a decidable witness of the state a decision was about, or a typed reason there is none; recompute and compare it before reuse; keep *no witness* distinct from *witness differs* to the consumer that acts. We show that content validity and lifecycle validity are orthogonal, each rule unsafe exactly where the other is sound; and that a time lease cannot separate a changed world from an unchanged one.

1 Introduction

A person approves an agent’s write to a file. Later the agent reaches a second write the recorded approval covers, and between the two a second writer has changed the file. The runtime dispatches. Measured end to end, **none of the gates on that path asks whether the approved world still holds** (Figure 1): the precondition gate asks whether the subject was observed at the session’s current revision, the authority gate asks whether a recorded decision names this capability and these resources, and the staleness guard asks whether the provider’s revision has moved. Each returns a true answer.

That agents act on stale state is known, and we claim no priority over it. It has a dedicated study [33], a correctness condition for commit-time authorization [38], and a systematization that names it a root cause: authorization *checked once and trusted forever* [8]. Prior work measures the failure from *outside* a runtime and trusts the signals the runtime emits. **We measure inside**, and find the signals are emitted, well formed, and not load-bearing.

Our claim is about why a runtime that checks can still



Figure 1. An approval recorded against W_0 authorises a write dispatched against W_1 . The runtime observed W_1 ; each gate returns a true answer and none asks whether the approved world holds. A content precondition checked at dispatch does not save it either, and §5.3 measures why at two commits: first because the dispatching run never looked, and then, once the runtime looked before every change, because it looked **after** the change — a valid basis of the wrong world. **Only a witness recorded at the decision sees the difference.**

miss. A human approval is a *memoized decision*: an answer to a question, reusable exactly when the question is the same question. A question is individuated by its *subject* — what was asked about — and its *premise* — the state it was asked against. This runtime records the subject only coarsely — capability and resources, not the proposal the person was asked about — and the premise not at all, so a later run has nothing to re-check. Memoization with a validity key is standard; **the claim is that human approvals are memoized decisions, and that this runtime keys them without the premise.** A reading of five other runtimes finds none recording a premise on an approval; that is a statement about what they represent, not about what they do (§6).

Contributions.

- **The anomaly, measured inside the runtime**, with two causes that share no input and a control that rejects the general reading (§2).
- **A model** — approvals as memoized decisions — with a checkable property and a three-field minimal state, and the measured claim that validity is one of *two* orthogonal primitives (§3).
- **Consequences derived from the measurements**, and three forward claims the design makes that future arms

can refute (§3.2).

- **Four places where building it contradicted its own reviewed design** — each a property of a representation rather than of a procedure, and none surfaced by static review (§4).
- **A controlled comparison** against published validity rules and two shipping frameworks, reduced to their predicates (§5.3), the cost of the mechanism, and one approval task run with a real model on the runtime, in which the witness recorded at the decision and the basis taken at the last look each refuse exactly the stale write the other cannot see — seven traced runs, not rates (§5.7).

What we do not claim. Not the phenomenon of agents acting on stale state, which has a dedicated study and a systematization; not the correctness condition for authorising at the point of commit; and not occurrence identity, which is formalised and benchmarked elsewhere [9]. Where our result overlaps published work, the contribution is the measurement and we say so at the point of claim.

2 The problem, measured

Terms, fixed once. A *world* is the state of the resources a runtime can address; this paper’s is one filesystem root addressed by two providers. A *resource* is an addressable element of it. An *observation* is a runtime-taken reading of one or more resources, carrying an identity for each — the only channel through which the runtime learns the world. A *premise* is the state an action is taken to rest on, recorded as the identities an observation returned, or as a typed reason no identity was available. An *approval* is one person’s recorded answer to one asked question. An *occurrence* is the concrete proposal that question was about, as distinct from the question’s *class* — the capability and the resources it names. An *effect* is a dispatched mutation. *Authority* is entitlement to act, which the runtime decides by matching a recorded approval; *validity* is whether a matched approval still rules. **These two are separate throughout, and §5.3 measures that they are orthogonal.** A premise’s *basis* is the pair it records — which resource, and the identity observed for it — and a *witness* is one such identity, the thing recomputed and compared before reuse. No term is used below in a second sense.

Setting. A person answers escalations and the runtime records each answer as a durable decision; intents are scripted, and no arm drives the runtime’s model port (§5.1, §7).

An approval outlives its world. A decision recorded against one world is matched and reused in a later run observing another, and it authorises the dispatch; the arm in

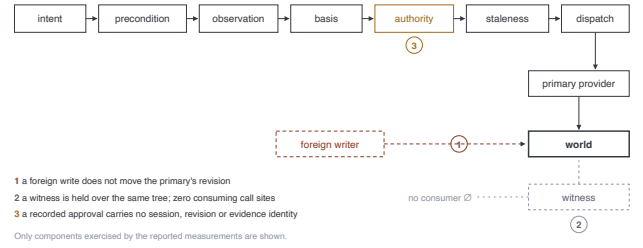


Figure 2. The path from intent to effect and the three seams where measurements landed: the precondition gate, the authority gate, and the staleness guard.

which an unrelated file changed and the arm in which the approved file changed agree on every observable. **With a declared premise the runtime looks, records the rewritten content, satisfies its own precondition with that fresh observation, and acts on the old approval anyway.** Declaring a premise cannot help by specification: the gate asks whether the subject was observed *at the session’s current revision*, not whether what was observed still supports the action. The behaviour survives the file-backed persistence path: after a fresh host reloads the journal the approval still authorises, while a premise derived from an observation is refused as unevaluable because its session is gone.

Across a journal reload. The persistence path writes one journal to disk; a fresh host reloads it, and the second pass is asserted to load from disk rather than memory (Figure 3). In one run the two decisions are treated oppositely: the resumed run cannot use the persisted observation and looks again, while the approval is not revalidated. A control in which nothing changed confirms that a resume alone does not invalidate an approval, which constrains any repair: it may not simply expire every approval at every reload. **On the daemon path, this reload is the answer’s second run within one turn.** The daemon keeps one journal per turn, so no approval reaches a later turn, and a pending approval is held in memory and ended by a daemon restart (measured on the daemon, in the restart arm of the end-to-end column).

Two causes, one verdict. The same permissive verdict is reached by two routes that share no input (Figure 4). On the authority path the dispatched action carries *no basis*, so the staleness guard exits on its first branch and no revision comparison occurs. An action carries a premise exactly when its proposer declares one; the declaration is the sole source of the basis’s targets, and observations contribute none. Where a premise is declared, the guard computes drift from a counter authoritative for one provider’s own effects; a second writer moves no counter, so drift is zero and the guard returns *before* it reaches the freshness mode —

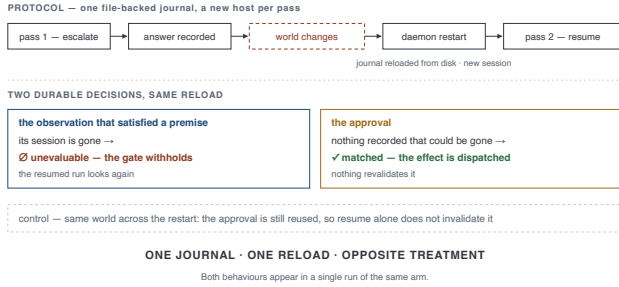


Figure 3. The persistence protocol. The observation-derived premise cannot be reused after reload because its session is gone, so the run looks again; the approval, having recorded nothing that could be missing, still authorises.

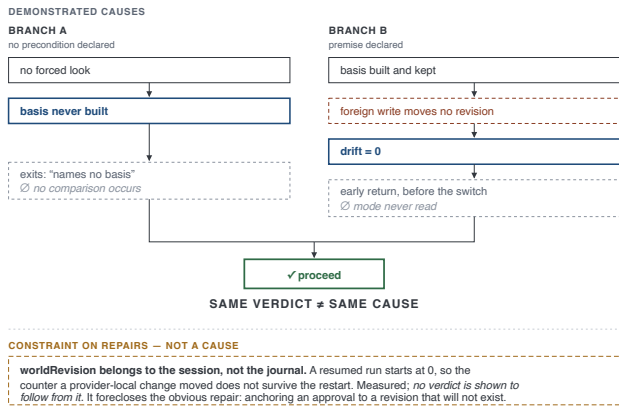


Figure 4. Two causes reaching one permissive verdict and, shown separately below the rule, a measured fact that is *not* a cause: the counter reset across a resume, which constrains the repair but produces no verdict. Observing verdicts alone cannot tell the two causes apart.

strict, revalidate and advisory behave identically on this path. **Counterfactually** — a real component under a mode this runtime does not produce — the enforcement repair the system had already chosen is inert as specified: with both halves implemented the verdict is `proceed` and its resolver is invoked zero times, while a control at forced drift invokes it once and refuses. The decision states two conditions it believes jointly sufficient; a third, that anything reaches the switch, is unstated, and could not have been stated, because the measurements establishing it came later. **Every gate keys on something a foreign change cannot touch** — a counter it cannot move, or a premise that was never built.

A measured fact that is not a third cause. A resumed run opens a new session at revision zero, so the counter a provider-local change moved does not survive the resume. We keep it out of the causal analysis: no arm shows a verdict following from it, and the positive reason is that the resumed run *re-looks* — the previous observation is `unevaluable`

because its session is gone, so the new look builds a basis at the current revision against a world that already contains the change. Its consequence is for the design (§3): a revision cannot carry validity across a resume.

The runtime detects the change and discards it. A working revalidator already exists on the path every observation takes: it compares a new observation’s facts against the identity a task depended on and returns `restored`, `invalidated` or `disputed`. The runtime observes the approved file move from one digest to another — and then returns early, because nothing a *task* depends on changed, and an approval is not a task. Two controls fix the cause to dependency rather than detection: a task depending on the changed resource is revalidated, and a task depending on a different resource has the same change reported and still discarded. Across a resume the detection is not even made, for an unrelated reason: the detector is a per-loop field, so a resumed run begins with nothing to compare against, and a control confirms that the first observation after a resume reports nothing. These are two mechanisms, not one seen twice, and neither is an absence: the detector is reachable, consumed and correct, and its output is dropped by a filter encoding a narrower notion of what may depend on the world than the runtime has.

A look that says nothing reads as nothing moved. The dependency tracker skips any fact that carries no identity and returns the value it returns when nothing changed — documented as *the world did not move*. An identity-less fact does not even establish a baseline, so a later observation that does carry an identity reads as *first seen* rather than *changed*. The layer above refuses exactly this conflation: the revalidator keeps a third verdict for *could not check*, and its own documentation says that folding it into *it is fine* “is how a verification plane stops meaning anything”. **The runtime preserves the distinction in one module and destroys it in the module that feeds it.** The path is production-reachable. File reads carry a digest and directory listings do not, because a directory has no content digest: one listing produced three facts of which two carried identity, a deeper one thirteen of which seven did. On the writing side, `fs.write` yields two facts, both with identity, while `fs.mkdir` and `fs.delete` each yield one with none. We first read that census as showing that a deletion cannot be revalidated. That is backwards: a receipt describes the world *after* the effect, and binding happens before it. Measured directly, deleting an existing file binds like any other write; what cannot bind is a *directory*.

The window is structurally zero. Where a premise is declared, the gate withholds dispatch until it is met and routes the look at that moment, so the look is the last event be-

fore the dependent action and unrelated effects land before it. Drift at a declared dispatch is zero by construction in this scheduler: the check does not fail by timing, it has no signal to act on.

A matching failure one plane above. Before any revalidation runs, the gate matches a recorded approval to an effect on equality of (*capability*, *sorted resources*); the resource list is mined from parameters by argument name, and arguments are excluded from the key by design. An effect whose parameters name no location yields an empty list, and two empty lists compare equal. Measured through the real gate, an approval to run `npm test` dispatched `rm -rf src`, and an approval to commit “*fix the parser*” dispatched “*rewrite history*”; both verdicts carried the person’s own recorded answer, applied to something they were never shown. The shape is concentrated, not general: of six write capabilities outside the filesystem, four present exactly the resource the effect is about. The command-spawning capability is not composed into the running daemon, so its arms are a property of the gate rather than a live path; the repository provider is composed, and a commit declares only a message, so the shipped instance is a commit. A partial rescue rests on prose: the list begins with the intent’s free-text target, so two commands a model described differently do not match — measured, and not a safety property. Every defence reported here runs *after* the match, so revalidating a wrongly matched approval changes nothing. The divergence between an approved and an executed action is named [40] and a canonical action identity formalises a fix [9]; what we add is that it occurs with no adversary and a truthful record — violated by a match rather than a lie — and, on the shipped daemon, that it is not confined to effects naming nothing: a key without the proposal’s content matches a different proposal for the same resource, which the user has ruled a failure (§5). §5 reports the repair of the empty case.

Complete mediation is satisfied, and that is the point. Every gate is reached, runs, and returns an answer [36]; no result here escapes a monitor. The failure sits one layer inside the principle: *check every access* does not say *against what*.

3 Design

Statement. A human approval is a memoized decision whose validity key is the state the decision was about. It may be served again only when that key can be recomputed and matches; and *key not recomputable* is a distinct outcome from *key differs*.

Three records, and what each carries. The runtime carries three durable decisions across time (Table 1, Figure 5).

Table 1. Three durable decisions, as recorded when measured.

	premise	approval	evidence bound to a criterion
records	session, revision, evidence	capability, resources, decided-at	session, revision, observation id
reports	satisfied / unsatisfied / unevaluable	matched / not matched	—
world gone	withholds, why	says cannot detect the question arose	rebinds; run does not look

An observation-derived premise records session, revision and evidence, and reports *unevaluable* when its world is gone. An approval records capability, resources and a timestamp that nothing compares against a clock, and cannot detect that the question arose. Carrying enough to decide whether a justification still rules separates those two; the third record shows it is not sufficient. Evidence bound to a criterion carries the *same* provenance the premise revalidates with — session, revision, observation identifier, each asserted present on the record the measured run bound — and on the path that rebinds it in a later run **none of the three is read**. So the condition has two parts: a record must carry enough to determine whether its justification still rules, **and something must ask**. The need to withdraw a durable permission is old [21, 22, 23]; what the contrast adds is its *trigger*.

A third condition: what is recorded must be comparable. The premise’s basis pairs a resource with an identity. The rebound evidence names a resource and carries none: a fact is minted with an identity only when a digest accompanies the path, and the look that answers a criterion returns a path without one. A different look from the same provider does yield identities, so the cause is the shape of the look, not the provider. **On this path a later observation can replace a justification and can never confirm one**, and the two obvious remedies trade off: repeating the look yields another incomparable hit, while the look that yields an identity answers a different question — *what is this file’s digest* rather than *what does its content say*. The split runs between branches of one function: a listing carries a digest and mints no evidence, an inline read mints evidence without one, and a withheld read mints evidence with path, digest and length together. Both modes are reachable, so a recovery mechanism must tell a justification it confirmed from one it replaced. Scope: one provider, two capabilities, three branches; how often a real goal takes each branch is not measured. The property at the end of this section was formulated after these conditions were measured, and its first two clauses restate them.

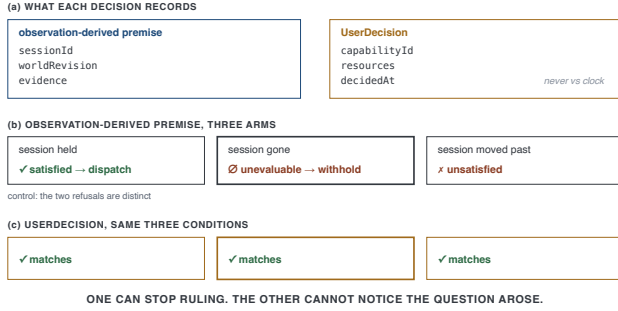


Figure 5. Two decisions the runtime carries across time. The one anchored to an observation can cease to rule; the approval records nothing that could be gone.

Eliminating the anchors. Four anchors could carry an approval’s validity across time, and measurement rules out three. A *revision* cannot: the counter belongs to a session and does not survive the resume that would need it, so a comparison of revisions across a resume cannot see even a change the runtime made itself in the previous pass. A *session* cannot: anchoring to one would invalidate every approval at every resume, which the same-world control shows must not happen. The runtime’s *existing identity memory* cannot: it is task-keyed and per-run, which is why it discards the detection it already makes. What remains is a **resource identity recorded on the approval itself** — in the model’s terms, a key. It is an elimination rather than a preference, and the survivor arrives with a measured constraint: it must distinguish *the identity is the same* from *there is no identity to compare*, because the layer beneath returns one value for both.

Coverage is gated by premise establishment. A key exists only if an observation was taken at approval time for it to be derived from, and one is taken only when the proposing intent declares a premise. Nothing in this runtime requires a proposer to declare one, and arms that declare none dispatch with no basis at all. Key-based validity is therefore structurally unavailable for any approval whose intent establishes no observation. We measured no workload and say nothing about how often that occurs; the gate itself is what is established.

Two primitives, not one. Validity answers *is this the same question?* A second, independent primitive answers *am I still entitled to act on the answer?* — the territory of capabilities, revocation and lifecycle-bound monitors [37, 21, 23]. **§5 measures that the two are orthogonal:** each rule is unsafe exactly where the other is sound. A model claiming one unified primitive would contradict that measurement. Ours is a single primitive for *one* axis, plus the measured claim that the other exists and is independent.

Minimal state. Three fields suffice:

```
Approval = { subject, key : Witness |
             NoKey(reason), verdict }
```

The shipped record has eight. Provenance of the key is not part of the key; the key is snapshotted when the answer is computed, so the reference moment is definitional rather than a field; and a spent-bit is needed *only* in the NoKey branch, where sameness is undecidable. Under the user’s ruling the *subject* is the concrete proposal the person saw; the shipped subject omits its content, and §5 measures the consequence.

The property a second implementation must satisfy. *For every durable authorization, record a decidable witness of the state the decision was about, or a typed reason no witness exists; recompute and compare that witness before each reuse; and keep no witness distinct from witness differs all the way to the consumer that acts.*

Where the design must choose rather than measure.

The verdict is three-valued, and both non-reuse outcomes escalate to a person rather than refuse: recording *the person was never asked* as *the person said no* would repeat, one layer up, the reading of an absence as a value that the look beneath makes. Two questions remain open and neither is answerable by measurement: what to do with approvals already recorded without a key, and whether to spend an observation when a person is asked, so that a key exists to record at all. The second is the precondition for the mechanism covering anything, and it arrives as a price that bookkeeping does not remove. The runtime’s budget contract is purpose-blind — a count of observation events, not of observations of a kind — so an observation taken to authorise falls inside it by construction, and exempting one means amending a stated contract. A verification look once reached a provider while uncounted; it was treated as a defect and repaired. A look taken to verify is still a look.

Obligation follows what is destroyed, not what the mode is called. The freshness modes were defined around whether a *position* still resolves, and that framing does not reach the capability its own documentation named as its example. A path-addressed write resolves its target by name, so nothing about position can go stale — **but what a whole-file write risks is not that its target moved; it is that it destroys content the caller never read.** Addressing by name makes the target resolve and says nothing about whose bytes are being replaced. The correction is not to redefine the modes but to notice that **one mode obliges two capabilities differently, because they destroy different things:** a patch destroys the region its *find* names, so *re-validate* means that region still resolves and an edit elsewhere is genuinely irrelevant; a whole-file write destroys the file, so there

is no region to preserve and any known change since the basis means it is about to destroy something nobody read. **This is why enforcement is a property of a capability and not of a provider (§5)** — the obligation follows the loss, not the name. *Derived by the runtime track while implementing the change.*

Two collapses of the third value, in this runtime, in opposite directions. The claim that *no witness* must stay distinct is not a design preference here: the provider’s own freshness modes are two ways of collapsing it, and they collapse it opposite ways. Its check resolves to *unchanged*, *changed*, or *uncovered* — the last returned whenever the basis observation does not carry the path being written, which is the ordinary case of reading one file and writing another, because the basis is the session’s most recent observation rather than one taken for this effect. **The strict mode refuses on anything but *unchanged***, so it reads *I have no witness as the witness differs* and declines a write it knows nothing about. **The revalidate mode refuses only on *changed***, so it reads the same absence as *nothing moved* and proceeds. **Neither can express the third answer, and each is unsafe exactly where the other is useless** — one manufactures refusals, the other is silent precisely where it has no information. The approval layer keeps the value: an approval with no identity to compare is *unevaluable* with a named reason, and that reason reaches the consumer that acts. **So a runtime can hold the same information at two layers and lose it at one of them**, and which layer keeps it decides whether a person is asked or a write is guessed at.

The witness convention already has a second consumer. The identity an approval records is not a representation built for authorisation. The runtime’s recovery path now uses the same convention — a per-resource digest harvested from what an observation reports — for a different question: a request refused with a disposition that forbids re-presenting it is not dispatched again, unchanged, to a world in which none of the resources it addresses has been *observed* to change; any look that witnesses a different identity for one of them unlocks the retry. **The convention is shared and the store is not:** the approval’s re-judge compares against only what its own covering look reported, while the recovery guard compares against the identities the runtime has accumulated. We cite this as evidence that the primitive is general, not that the two consumers agree, and the recovery guard’s own measurement includes a firing in a test no one had pointed it at, found by logging every firing across the suite rather than by the suite’s assertions.

3.1 What the design forbids

3.2 What the design forbids

Every consequence in this section was derived from measurements already taken; none was predicted by the model before it was formulated. The model was stated after the results it explains. We therefore lead with what it forbids, which future arms can test.

Three forward claims. (i) *No second separating cell:* a plan-time and an approval-time key may differ only where the state moved between plan and approval. (ii) *No load-bearing provenance link:* any observable that depends on how a key was computed refutes the model. (iii) *No consumption tracking where every resource has a witness:* a spent-bit is required only when no key exists.

Consequences.

- **Mutation invalidates; revalidation is recomputation.** A changed key component is a cache miss, and revalidation is not a mechanism but the definition of serving a memo safely.
- **In the measured arm, a fresh read did not establish sameness.** The runtime took a fresh look and satisfied its own precondition with it; a fresh read yields only the *current* value, and sameness needs the *stored* key. We state this for the arm measured. Whether fresh validation in general falls short of approval-relative validation is not tested here, and we do not claim it.
- **A cache with no key cannot report a miss, only a hit.** This is the form of a key-related failure on more than one plane: an identity memory that treats a fact with no identity as unchanged, and a freshness guard whose key omits a component a foreign writer can move. The model accounts for the *form*; it does not account for which planes fail or how many.
- **Single-use is forced, not chosen.** Without a recomputable key sameness is undecidable, so an answer covers only the asking it was given for. The shipped runtime applies it only to a binding that names a resource without an identity; an effect that names nothing records no binding and is reused unchecked (§7), which this consequence forbids.
- **Content alone cannot be sufficient.** Closing the episode that justified a grant, with content unchanged, leaves the question the same, so validity alone must reuse the answer. The content rule’s 14/14 unsafe permits on closed episodes (§5) are this consequence rather than a defect of the model, and they are why a second primitive is needed (§3).

- **Provenance is inert.** A key does not include how it was computed; §5 measures the link to the observation a person saw as carried and unread.
- **Reuse costs what first use costs.** Establishing a key and re-testing it are the same operation: obtain a current witness per component. §5 measures this on counterfactual inputs, and its replication in the shipped mode is pending.

The model does not explain the distribution of failures across layers, and it does not claim the phenomenon, which is externally named.

4 Implementation

The mechanism's design was written down and reviewed before it was built, and building it contradicted the design in four places. None is a coding error. In each, the design held a belief about how the runtime *represents* something; the procedures were written correctly against that belief; and the built runtime represents it otherwise. **Static review surfaced none of the four**, because a review reads procedures and each of these is a property of a representation.

What was built. The surviving anchor was implemented: an approval now records the identity of each resource it was granted over, taken through the loop's single gated door, and is revalidated before reuse. A changed identity escalates rather than dispatching, and *I cannot tell* stays distinct from *it changed*. An approval with no identity to compare takes a no-identity branch that treats the person's answer as covering the use it was given for and no later one: it dispatches on first use, and on any later use refuses as *unevaluable*, naming the missing identity. The control that constrained the design holds: a resume alone does not invalidate an approval.

Authority and evidence are not independent planes. The design assumed that deciding what an effect may touch and observing what it is about are separate concerns. The shipped envelope granted nothing, so the runtime could not observe what it was asking about; the approval recorded nothing, and the failure presented not as a refusal but as a person answering an approval and being asked the identical question again. A runtime required to identify what an approval is granted over must be permitted to observe it; *grant nothing* is not a safe default for such a runtime.

One resource list cannot carry two semantics. Permission and binding first ranged over one list, which begins with free text a model wrote — in the measured case ["notes.txt in the workspace root", "notes.txt"]. Over-inclusion is correct for a fail-closed scope and fatal for binding: the prose entry cannot be identified, and a correct binding of the real path was reported un-

evaluable because of it. They now range over different lists: a separation of semantics, not a data-cleaning fix.

Consumption is a fact about a dispatch, not a run. The implementation read *a later use* as crossing a run boundary, and an approval is always consumed in a run after the one that asked, so the single-use rule was a zero-use rule. Which approval permitted which dispatch is not derivable from run structure at all — and the event recording a permitted action carried no authority, while refusals carried a full report. The system could say why it declined and nothing about why it proceeded.

Not reported is not absent. The design assumed that a look which says nothing about a path establishes that the path is not there. A look reports what it examined and nothing about what it did not; inferring absence from silence requires a complete look, and nothing establishes one. In the model's terms, a key that could not be computed was read as a key that matched — the conflation of *could not check* with *it is fine*, reintroduced inside the mechanism written to refuse it. **A green suite of 44 pins and a seven-mutation battery both endorsed it:** every fixture's look happened to cover its own target, and a listing of the root covers a file in the root but never one two directories down. A pre-existing test had ruled on this exact failure mode in principle, one layer down, and did not go red. **A fixture suite written at the root agrees with this bug.** It was found by printing the value — a file just written to disk bound as absent — and an arm with a deliberately target-ignoring port and a nested path was added afterwards, with an eighth mutation to convict it.

The instruments both succeeded and failed. Two mutations survived the first battery run, both of one shape: the mechanism would have been *declared*, *wired*, and *inert* — never producing a single identity — while every pin stayed green, because an approval bound without an identity still works once. A second test exists to convict them, and one of its own arms initially had the same defect. That is the positive case; the negative one is the fourth contradiction, where the green suite and the passing battery both agreed with the defect. We report both, and do not rank mutation against reading as a way of finding defects: the count is too small to rank.

5 Evaluation

The evaluation is ordered by the questions a reviewer will ask of it, not by the order the experiments ran. §5.1 states the conventions every number below obeys.

5.1 How every number here was produced

Arms marked *daemon* run the shipped binary, answered by a real client. Every other arm runs production runtime code under a test harness (the real loop, coordinator and journal, with scripted intents), and arms marked gate-level hand the gate an action built by hand. The substrate is two providers over one filesystem root, one machine, a single writer, and scripted intents. Each experiment registers numbered predictions before measuring and keeps the refuted ones verbatim. Of the 272 arms, 27 measured something other than their written expectation and are retained, each typed by whose belief failed: 23 a pre-registration, 3 a design document, and 1 held by the implementation and written down nowhere. Counted by prediction rather than by arm they are 18, because one refuted cost prediction spans ten cells; instrument predictions, such as one about a baseline, are recorded separately and are not in these counts. A further 11 arms were predicted by no one and carry no match verdict, since without a prediction there is nothing for a measurement to match. Each ships a mutation battery, and a surviving mutation is answered as an *untested branch*, a *weak assertion*, or *behaviourally identical*. Every result is a test in the repository, and each of the 272 arms names the commit it resolves in, in a ledger from which this paper's tables are generated; a check in the repository verifies that each recorded commit touched its experiment.

Comparisons run in two columns. Where a mechanism is compared with its absence, both columns run the same scenarios through the same harness. The baseline is the same tree with one mechanism removed at a single site — never a historical checkout, which would confound the repair with everything else that changed. Each scenario's correct outcome is labelled before measurement, and an escalation on uncertainty is counted in its own row rather than folded into either direction. An edit that fails to land would make every cell agree and read exactly like an inert repair, so each baseline edit is refused unless it lands at exactly one site; and a scenario carrying no decision must escalate in every column, or that column cannot report a block.

Evidence classes. A result is *observed* in one experiment or *replicated* in more than one, with the count stated; a *refuted* prediction is retained rather than removed; and a *counterfactual* evaluates a real component under a mode this runtime does not produce, and is not evidence of runtime behaviour. Most results here were measured once. The two causes of §2 are replicated (§8), and three results are counterfactual and marked where they appear: the enforcement repair of §2, the inputs of the cost measurement, and one patch arm's binding (§5). Two compile censuses put every action in every experiment through the shipped compiler's schema check, after first confirming that a known-bad input

is refused. The anomaly and the journal reload compile as shipped. Refused inputs are confined to three experiments: the cost measurement, one patch arm, and the repair's gate-level comparison.

What the numbers are. Counts here are of five kinds, named wherever one could be read as another. An *arm count* is a property of experimental design; a *control* or *mutation count* is a fact about the instruments; an *observation-event count* is something a real execution produced; and a *derived figure* is computed from a rule and labelled as derived where it is used. **No number here is a workload statistic.** We ran no workload, so nothing states how often anything happens in a running system: that nine arms followed one causal branch is a fact about this study, not evidence that most approvals lack a premise.

5.2 RQ1: does the mechanism capture the state it claims to?

Correction without over-invalidation. Twenty-six scenarios run in two columns — the built runtime, and the same tree with one mechanism removed at a single site — each labelled before measurement as a reuse that ought to stand, one that ought not, or an escalation on uncertainty. **All fourteen reuses labelled ought to stand still dispatch, and nothing moves in the permissive direction.** The zero is falsifiable: injected over-invalidation fails eight of twenty-nine cases. Seven of the eight are cells that existed before the same-world control was added, so that control extends the check into a world where nothing changed rather than being what makes the zero readable. **Fifteen of the twenty-two stand and fall cells hand the gate a write the shipped checker refuses.** For nine, outcome, rule and route are unchanged once content is added **in both columns**, so their movement between columns is measured on a producible input; six rest on inputs no compile produces. The five loop arms and two commits compile as shipped. The daemon measures the top-level case (§5.4). Four end-to-end arms cross *who wrote* with *what was written* (Table 2); both writes to the approval's own resource escalate with an identical recorded reason and both elsewhere are reused, so **the repair is decided by which resource moved, not by who moved it.** A control approving, resuming and writing nothing is reused in both columns. A mechanism that refused everything is not uniformly worse: it fails the fourteen stand cells and is right about the one cell both columns get wrong — an effect whose only resource is prose, which neither repair reaches. The comparison therefore does not license *the repair dominates*; it licenses a statement about which cells each gets right.

Provenance is not read. The binding records a link to the observation its key came from. Six arms through the real

Table 2. The repair turns on which resource moved. Four end-to-end arms at a persistence-and-resume boundary; every baseline dispatches. Self-caused: through the runtime’s coordinator; foreign: behind the runtime.

	approval’s own resource	unrelated resource
self-caused	escalate	reuse
foreign	escalate	reuse

loop, coordinator, journal and notification adapter differ only in that field — present, omitted, and pointing at an observation that does not exist — and **every observable is identical in both world states**: the stop reason, whether a question is raised, the authority events, the observations spent, the projection, the sentence a person is shown, and the revalidation verdict. The pre-registered falsifier was any observable differing; it fired zero times. The dangling link is the sharpest arm: a field that is merely carried cannot notice a broken link, and a field that is resolved must. The field is produced, persisted, loaded back, and dropped at the point of use, so we report it as *not reachable* rather than *absent* — and against ourselves, since the field’s rationale was that a durable approval should name what the person was actually shown. The mechanism binds to a *value* at the approval moment, which is structurally what optimistic concurrency does with a read-set version [30]; what separates it is *when* the value is captured relative to a human decision.

5.3 RQ2: is it distinguishable from a weaker alternative?

This is the section that decides the paper. A reviewer’s question is not whether the mechanism works but whether something already published accounts for the same results.

Against published validity rules. We reduce three published designs to their validity predicates and evaluate them on our scenarios: a lifecycle rule keyed on whether the justifying episode is open [37], a fixed lease [5], and a plan-time reference checked before execution [7]. These are validity semantics, not the systems, which were not run.

Two shipping frameworks, run. Two frameworks unmodified from PyPI, pinned, in six configurations a developer could ship: LangGraph’s resume with the plan taken before the pause and with the node re-executed, and the OpenAI Agents SDK’s approval under its default fingerprint, a changed call, a standing grant, and a developer-written `needs_approval` callable. Sixteen arms. **Every one executed and none declined on a changed world**, so *among frameworks* the mechanism is alone in refusing relevant drift and alone in re-asking on irrelevant drift. **Whole runtimes**

Table 3. Unsafe permits, 14 cells per arm. Content and lifecycle binding are orthogonal.

	episode open, content changed	episode closed, content same
content (shipped)	0 / 14	14 / 14
lifecycle	14 / 14	0 / 14

do not support that sentence (§5.3). One framework executed a proposal its user never saw: approval shown `retries = 3 -> 5`, the world drifted to 4, and `retries = 4 -> 5` ran — the seventh bin, in software we did not write, and the class of §5.5, so **instance binding is not an artefact of our implementation**. The SDK’s invited check was **evaluated once and never re-asked**, so a developer can write the witness and nothing asks it again. Each comparator is right about something: one re-pauses on a changed call, the only refusal in the column, and one never runs a stale plan. **This and §5.3 agree because they probe different effect shapes.** A find/replace patch *carries its own content precondition* — its `find` stops matching once the text drifts — so a provider refuses with no approval re-checked; a whole-file write carries none, and the compiler stamps every basis *advisory*. **So the mechanism is unique on effect shapes whose content is not already a precondition, and duplicative plus one re-ask where it is.** *Bounds: frameworks, not runtimes; a scripted model; two versions. Every effect applied here is a whole-file write, in the comparators and in our own arm, so a patch is only what a person is shown; the comparators were given no find/replace tool, the configuration where a content precondition would do this without them.*

Each rule is unsafe in exactly the cells where the other is sound (Table 3), and we report both halves: the shipped rule permits reuse after the justifying episode closes, because content is unchanged and content is all it consults. **A lease cannot separate a changed world from an unchanged one:** a content-blind rule receives identical inputs from the mutated and control arms, and no pair of the forty-nine lease-by-age cells differs, so it cannot reach zero unsafe permits and zero unnecessary escalations at any setting. (Its cost is then constant across settings by construction; we report that as arithmetic, not as a result.) A plan-time reference agrees with the shipped rule in every cell but one (Table 4) — where the world moves between plan formation and approval, it re-asks a person about a change they have already seen and approved, nine unnecessary interruptions against none. The re-validation proposed as a unifying remedy is therefore not neutral: it manufactures interruptions.

A patch, and a traditional technique that beats it. Asked of `fs.patch` on the daemon, with a wording that routes by construction. An irrelevant foreign edit makes the re-

Table 4. Plan-time reference against the shipped rule, 9 cells per arm, as (unsafe permits, unnecessary escalations). The two differ in one configuration only.

scenario	shipped	plan-time
no change	(0, 0)	(0, 0)
changed before approval	(0, 0)	(0, 9)
changed after approval	(0, 0)	(0, 0)
episode closed	(9, 0)	(9, 0)

pair **re-ask, an unnecessary rejection** — its witness is a whole-file digest — while the *counterfactual* column without the re-judge **applies the patch**, and refuses the relevant edit through the provider’s own exact-match guard. **So for find/replace patches a content precondition the provider already has catches this drift, and the mechanism costs one re-ask and one look.** Predictions held in both columns, and the file’s digests carry them.

Two shipping runtimes, run. Two closed-source agent runtimes, unmodified, each driving a real model: Claude Code 2.1.269 and Codex 0.149.0. Both expose a pre-tool hook with the same wire shape — it receives the call the runtime is about to make and returns allow or deny — which is the programmatic seat of the approval decision, so a hook standing in for the person does what §5.4’s arm does: record the approved proposal, change the world, allow. Codex’s contract is read from its source at the installed version. Eight cells, two effect shapes each, with controls.

Both runtimes’ native editing tools refuse a proposal formed against the pre-mutation world: one will not save a file that changed after its read, the other’s patch fails on a context line that no longer matches. **Neither refusal is approval revalidation** — both are content preconditions, the class of §5.3, and neither consults any record of a decision. **The same runtime, the same approval semantics, one command rewriting the whole file: both execute the stale proposal.** One approved and ran `printf 'retries = 5\n\ttimeout = 10\n' > config.ini` into a world that had moved to `retries = 4`. **So the discriminating boundary is the effect shape, not the runtime**, and it runs through three independent implementations at the same place: ours declares a content precondition on `fs.patch` and none on `fs.write`, which is read off the capability declaration rather than inferred from a run.

The bound is the whole point of stating it here. **The hook answers every call, so neither runtime reuses an approval**, and this column is therefore silent on the paper’s actual subject: it measures whether a *dispatch* is stale, not whether a *decision* is. A hook is also not a person, one runtime’s model was not captured, and each cell ran once.

Two layers, not two competitors. A content precondition is an **execution-layer** guard: per capability, provider-owned, about the bytes, cheap, failing closed at dispatch. Revalidation is a **decision-layer** guard: cross-effect, runtime-owned, about whether a person’s decision still applies, and it must survive a restart. **A precondition cannot answer the decision question because it does not know a decision exists**, which is why §5.3 loses without contradicting the rest — on that one effect shape the decision-layer question is answered in passing by an execution-layer guard. **The falsifier, and it is satisfied by a case we had not named.** If no case exists where revalidation changes the outcome for the better, it is a more expensive precondition and should be deleted rather than defended. **The case we predicted — a precondition that passes while the approval is stale — is measured, and on it the mechanism loses.** The provider’s precondition is an exact *substring* and the identity an approval binds is a digest over the *whole file*, so the two overlap without nesting; an irrelevant edit leaves *find* matching while the digest moves, and there HEAD raises an **unnecessary rejection** where the column without the re-judge **executes correctly** (§5.3). On that shape revalidation is strictly a cost.

The case it wins is a whole-file write, and why it wins there changed twice. As first measured, a whole-file write carried no content precondition — its only prior-state constraint said the old content is *discarded* — so the gate was the only thing between an approved write and a stale one: **four arms escalate under the mechanism and execute stale in the column with the re-judge removed.** The runtime then gained a precondition for that write, and then began looking before every change, and the same cell kept the same two outcomes for a different reason each time. The removal column is what establishes each reading, rather than an argument. What survives all three versions:

Revalidation’s value is not that it catches drift a precondition passes — there it is strictly a cost. It is that it holds a witness from the moment a person decided, which a precondition checked at dispatch cannot hold however it is supplied, because that precondition’s witness is taken after the decision.

A boundary, corrected here rather than left implied. We earlier named the predicted case as an approval over one resource whose justification rested on a fact about another. **That case this mechanism cannot catch at all**, and the source settles it without an experiment: the resources an approval binds are harvested from the compiled action’s arguments and the intent’s parameters, so a resource the effect does not name is never bound and never compared. **The mechanism re-judges a decision against the resources the effect touches, not against the reasons the person had.** That is a limit on what §3 claims, not a defect, and we state

it because the stronger reading is the one a reader supplies for free.

We ran that experiment, and the answer is a third branch neither prediction named. The runtime already had a whole-file digest precondition, runtime-supplied rather than named by the proposer, switched off by a literal: the compiler stamped `advisory` on every basis it produced, **overriding the schema’s own safer default at every call site at once**, under a written rationale that a path-addressed write cannot go stale because its target resolves by name — true about the target and silent about whether the bytes being replaced are the bytes anyone read. Turning the literal into a parameter and giving the whole-file write the enforcement the patch already had, **no cell in the grid moved, in either column**, and that is not a null result.

The cheap guard was parasitic on the expensive one, at that commit. An action carries a basis only when the run that compiled it has taken a look itself; the grounding is drawn from that run’s own observations. In the column with the re-judge removed, the dispatching run’s events were a goal, a proposal and a dispatch — **it never looked** (every committed dump of that column before the next change reads zero looks for that run), so there was no basis, the new precondition had nothing to compare, and the foreign write was clobbered exactly as before. Under the mechanism the same run does look, and those looks are the mechanism’s own. **We reported that as a property of the implementation, not of the idea: a runtime whose dispatching run looked for its own reasons would hold a basis with no approval machinery at all.**

The runtime then became that counter-example, and the separation survived on better ground. A later decision made the runtime look before it changes something, for its own reasons: once authority decides to dispatch a non-read effect whose resources the run has not observed, it takes a covering look and regrounds the action on it. We re-ran both columns at the new commit against predictions committed beforehand, and all six held. **The ablation column still executes the stale write, and the reason is new.** Its dispatching run now does look — the recorded prompt identifies that look as the pre-change look, where every earlier dump of the column shows none — but it looks *after* the foreign write. Its basis is the changed file’s digest, the provider’s re-validation finds nothing moved, and the write lands with a fresh, valid basis. The approval had recorded the file as it was when the person answered. Under the mechanism, on the same cell and the same commit, the gate compares that recorded identity with the look and refuses; none of that column’s looks is a pre-change look, because its covering look already covers the resource.

A witness taken at dispatch — however reliably, and whoever causes it to be taken — is taken after the decision, and cannot see what changed

between the decision and the look. Only the witness recorded when the decision was made can.

That is the distinction we had pre-registered as open — *a precondition binds the call; an approval binds a decision* — measured on a cell that already existed, and it no longer depends on whether the runtime looks. The same pair of readings was later reproduced with a real model proposing, where a second cell also showed the converse: a change after the last look is refused by the basis in both columns (Table 6). It does depend on the window between decision and look being non-empty: when nothing changes in it the two witnesses agree, and the mechanism has paid its look for nothing (RQ5). *Bound*: one cell, a whole-file write to a top-level target, a scripted model, one foreign writer. The unattended mission of §5.7 measures the same window from the other side, where no decision was made and nothing therefore recorded what the world was.

Two results against us in the same run, recorded here rather than in a footnote. The change does *not* close the nesting bound of §5.4: a nested target under a look that does not name it still executes stale, because the basis exists but does not cover that path, and the mode chosen passes exactly that verdict. **The third value collapses again, one layer down and in the permissive direction.** And three further mutating capabilities remain unenforced — now a named gap rather than an unexamined one, since each destroys something different and inherits no answer from the whole-file case.

5.4 RQ3: does the position of the world change matter?

On the shipped daemon. Fourteen arms repeat the question through the shipped binary, its socket and a real client answering, with no decision seeded, against a column with the re-judge removed at one site. Replications gave identical gates and final worlds. A foreign write to an approved top-level file **between request and answer, or between answer and dispatch, is refused** by the gate, while the column without the re-judge executes the stale write; an unchanged world, an unrelated file and a write made before the request all dispatch. **A nested target the model’s look did not name executes stale over the foreign write** in both columns, because the look covered only the root; the same arm on a top-level file is refused, so the boundary is nesting. The patch workload first never reached a gate — its wording shared no term with any capability — and its utility question went unmeasured on this path. **A later routing change let a proposal’s arguments reach the one reversible capability that accepts them when the sentence reaches none, and the same three patch arms then reached the gate and matched every prediction registered for them:** with the re-judge, the control executes, a relevant foreign edit escalates

with the foreign text kept, and an irrelevant one escalates *unnecessarily*; with the re-judge removed, the control executes, the relevant edit is dispatched and refused by the provider's exact match, and the irrelevant one executes correctly over it. **So the patch result of §5.3 reproduces on the shipped daemon:** a content precondition catches relevant drift with no mechanism, and the mechanism adds an unnecessary re-ask. The change that made the arms reachable is a routing change and touches no part of the mechanism. The daemon serves no resume, and after a restart a person's answer to the old request is dropped without a reply. **No approval showed the content it approved** (eleven requests), so every *correct execution* here is judged against world drift, not against the instance semantics the request fails. The countable cost is one look per approved dispatch.

5.5 RQ4: can an approval be reused for the wrong thing?

A matching key that was empty, not wrong. The repair is narrow on purpose. Where an effect names a resource the coarse key is untouched; where it names none, a discriminator decides — the canonical form of the compiled action's arguments, which already existed and was already pinned, so the finer key is not invented here. It is set only when the resource list is empty and kept out of that list, whose entries reach scope rules and the sentence a person reads. The design argued that a person approving a write to a named file approves it as an effect, not as one occurrence; with an empty list that class collapses, and **the comparison is not wrong, it is empty**. The user has since ruled the argument itself wrong (next paragraph). No use limit applied — the single-use rule never reaches an effect that names nothing (read from source) — and one would not have been a repair: the wrong effect can *be* the first use, and in the measured arm the recursive delete dispatched under a decision never used before. The claim that the change reaches no effect naming a resource was stated so it could fail, and seven pins across six experiments about such effects stayed green and unmodified. One pre-registered expectation failed: two commands a model described identically still match after the repair, because each names the description as its resource. **The repair addresses effects that name nothing, not effects a model gave the same name;** the prose rescue is still load-bearing where it was, and the arm is kept as refuted with its wording unchanged. A decision recorded before the field existed carries no discriminator and matches nothing rather than everything, so the person is asked again rather than overruled.

An approval covers a question, not the proposal the person saw. Three arms ran through the shipped daemon, its socket and a real client, with no decision seeded. The request shows the capability, the resources and a fingerprint, **and no content**; its identifier is the `(capability, resources)`

key the match uses. With the world unchanged, a re-proposal that kept the target and changed the content **ran on the earlier approval** — the dispatch names that decision, revalidation found the unchanged world reusable, and the new bytes are on disk. An ordinary approval covers the concrete proposal the person saw, so this is a failure, recorded as *unapproved proposal executed*. **The phenomenon is named [40] and the fix is formalised [9]; we claim neither.** What is measured here is that it occurs with *no adversary*, under a matching key the runtime's own documentation defends — and that the coarse key is not the defect. The unstated premise that `resources` names the *to what* is.

It is repaired, and the repair is measured on the same daemon. The approval's identity now includes the proposal's canonical arguments, at three single sites — **a published and benchmarked fix [9]**, so what is ours here is the measurement and not the mechanism; the same three arms give **correct rejection** where the defect gave execution — the second write never reaches disk, counted off the file's bytes — and the arm re-proposing identically still dispatches unchanged. **One extra approval, charged to that cell alone.** It also invalidates *every* approval recorded before it, and the content now reaches the request twice, as the question id and as a parameter: **disclosure, not presentation, and no substitute for a request that shows what a person must read.** *Later, on the mission path only:* a terminal interface to missions now presents each question with the proposal's full arguments and the identity each resource is bound to, and one session with a real model was run through it by hand — asked, shown the patch and the file's digest, approved, applied, verified, completed; in that session's journal the approved patch was dispatched, in the run that continued over the asking run's journal, on a basis naming a look taken in that run. That is presentation on one path, observed once; the daemon's wire requests still do not show content. **And it reaches a defect its own scope did not name:** an earlier measurement had an approval to run one command dispatch a different one — both named a resource, so the coarse key applied — and the repair of that era declined the case explicitly. Instance identity closes it, the arguments differing, and the arm was built to escalate *for that reason* rather than because a legacy decision matches nothing, which is what makes the result readable. *Scripted re-proposal; the unrepaired comparison is recorded rows, not a re-run.*

5.6 RQ5: what does it cost?

Cost. Reuse buys the same covering look first use buys (Table 5) — at eight addressable resources, eight observations, tracking the resource count. The worst case, one observation per addressable resource, is realised under narrow looks; a directory listing carrying a per-child digest serves any number with one look. **The runtime contributes the covering loop; the multiplier belongs to the observation**

Table 5. Observations spent by addressable resources N , identical on first use and on reuse. Counterfactual inputs; see text.

N	1	2	3	5	8
narrow looks	1	2	3	5	8
per-child digests	1	1	1	1	1

source. These are counts, not latencies: no arm drives a model or makes a network call, so the cost that would dominate in production is not measured here. **The inputs are counterfactual.** Every arm proposed `fs.write` with an undeclared parameter carrying the extra resources, which the shipped compiler refuses before the gate; the code measured is shipped, and the channel that put N resources on one write is not. At $N = 1$ the input is equivalent to the shipped form, by argument rather than measurement, and a replication in the shipped mode is pre-registered; no shipped-mode measurement exists for $N \geq 2$.

Once the runtime looks before changing anyway, the mechanism’s look is no longer extra. The runtime now takes a covering look before any mutating effect whose resources the run has not observed — one model-formulated request and one observation, charged to the goal’s observation budget — so a delegated mission pays that on every uncovered write whether or not any approval exists. On the attended path the two looks are the same look, paid once by whichever takes it first. **Measured on the control cell at the same commit, the two columns spend the same number of looks:** two in each, where the mechanism’s column spends both on its re-judge and binding and the ablation column spends one of them on the pre-change look instead. So against a runtime that already grounds its changes, the mechanism’s countable marginal cost on an unchanged world is zero looks. *Stated as counts: the looks in this grid are answered by a stub, and a model-formulated look’s latency is not measured.*

The same look count hid a cost that is not a count: the shared look left approved writes with no basis. The pre-change look regrounds an action only when its resources are not yet covered, and the re-judge’s covering look happens after the action is compiled, so it covers the resources without grounding the action on them. Read from the committed journals of both columns at two commits, not inferred: **under the mechanism, every approved top-level dispatch the gate let through went out with no basis at all** — the unchanged-world controls of the write and the patch workloads, the irrelevant-change cell, the change made before the question, and the late answer acted on after a restart — while **every dispatch in the re-judge-removed column carried one**, from its own pre-change look. The only mechanism-column dispatches with a basis were the nested-target arms, where the re-judge’s look missed the nested path and the pre-

change look fired after all. So in this runtime, as measured, **adding the decision-layer guard switched the execution-layer guard off for every write a person approved**, and the window from the re-judge’s look to the dispatch was guarded by nothing. It is the inverse of the dependence §5.3 first reported, and like that one it is a property of an implementation: the runtime track found it while building the re-submission change and is repairing it by grounding an uncovered-basis action on the covering look, without moving an existing basis. **The repair is measured on the grid:** grounding an action that has no basis on the look that covered all of its resources, and leaving an existing basis where it is, every dispatch under the mechanism at the next commit carries a basis naming one of its own run’s looks — the controls, the irrelevant change, the change before the question, the late answer after a restart, and the nested arms — **and no cell’s gate or final world moved** in either column. A partial cover still takes the pre-change look. What the repaired basis protects, a change landing between the re-judge’s look and the dispatch, is not injected anywhere in this grid; that window is measured with a real model in §5.7.

Under a real model, exact occurrence identity re-asks.

The instance-binding repair of §5.5 costs one extra approval where a proposal changes. With a real model re-proposing an approved write in the next run, the proposal changes almost always even when nothing else does: in the four mechanism rows of the approval task every re-proposal differed from the approved arguments, and both unchanged-world rows asked the person a second time. The differences were optional arguments, not content. This is an unnecessary rejection with a new source — not the world, and not the proposer’s intent, but the proposer’s phrasing of arguments — and it is removed by re-submitting the approved proposal rather than by loosening the identity: re-run that way, neither unchanged-world row asked a second time (§5.7).

5.7 RQ6: does it hold inside a real agent runtime?

Partly answered, on one path, and the rest is the honest state of the work. Every result above this subsection is at approval granularity: one effect class, one machine, a single writer, scripted intents, and a daemon that served no resume: after a restart a person’s answer to the old request was dropped without a reply. **Half of that has since been closed and measured on the spawned daemon.** The arm that pinned the defect now reads differently in three of its six numbers — the answer is matched to the waiting mission by wire request id, a run re-opens over the journal holding the question, and **the write the person authorised is on disk, put there by a process that never held the run that asked. The other half did not move, and cannot. Re-issuing an outstanding question to the client that asked**

it is not expressible on this wire, however durably the answer is stored, and *two independent properties of the protocol each suffice*. A thread is owned by the connection that started it, so a `turn/start` naming a thread this connection did not start is refused by construction — the *same* daemon will not serve another connection’s thread. And a thread’s identifier is *derived*, from the provider, the session and the time the session opened, so a *restarted* daemon opens a new session and cannot mint the old identifier at all. They fail differently, and a design answering only the first would still lose the thread across a restart. A resume claim that includes the person seeing their question again is blocked on a protocol surface rather than on effort. **So the answer acts and the person is told nothing**, and we report the two halves apart rather than the one that moved as the whole.

And that arm does not test the mechanism, which is the distinction that matters most here. The re-opened run goes through the same resumption path the in-band case uses — deliberately, since a second path would be two implementations of consent — so it receives the *whole* mechanism, revalidation included. **Receiving it is not being tested by it:** nothing changes between the answer and the dispatch in that arm, so the approval was still valid and the gate had nothing to catch. **A guard that runs on a path is not thereby a guard that could have failed on it** (§8.1). The arm that would test it — a world that moves while the person is away — is not built, and RQ6 stays open. Nothing here measures a runtime that plans, delegates, verifies and recovers *over hours*, and no number here should be read as if it did. The protocol that would answer it is pre-registered rather than sketched — seven scenarios, each stated by its discriminating measurement and by what it would refute: a long-running mission, world mutation during execution, an approval that goes stale, crash and restart, parallel agents whose premises interfere, a partially applied effect, and a false completion an independent verifier contradicts. It is registered before the runtime exists precisely so that a passing run cannot define its own success criterion. What can be said today is how far the binding currently reaches.

An unattended mission, measured on the mission path. One arm now exercises what this paper’s grid cannot: a mission with authority *delegated* in its own contract, no client attached, and a foreign write landing while it works. **As first measured, it never looked.** The runtime observed for exactly two reasons — an unknown worth resolving, or an approval that had to be bound before a person was asked — and a mission that needs to ask nobody has neither. So its write carried no basis, the provider’s precondition had nothing to compare, the foreign write was overwritten with no refusal and no diagnosis, verification was satisfied by the acting provider’s own receipt, and **the mission recorded itself completed**. The same mission with the write withheld blocked, and the foreign write survived. **Looking was a side**

effect of needing to ask, so the authority that makes a mission autonomous was also what made it blind. This is the problem of §2 one level out: every component correct, the world destroyed, and the record reporting success.

It took three decisions to make that arm recover, and each exposed the next. Making the runtime look before changing something put a basis on the write. A foreign write landing *after* that look was then refused as stale — and **verified as success**, the mission completing with the other writer’s content on disk, because the verification rule turned a refusal into a failure only when nothing had been observed, and here the look happened to show the assertions holding. *Execution failed* and *the state matches* had been one verdict, and the refused action was credited with a world it did not make. Separating them made the arm recover without a person: refused, failed, diagnosed, re-observed, applied, verified, completed with the mission’s own content. The same fold had been guarded by the test suite as correct behaviour: two routing tests asserted a completed goal whose search had been refused and never run. **A foreign write landing before the look is still overwritten**, and is by design: the look grounds the action on the world it finds, not the proposal on the world the proposer imagined. That residual is the window §5.3 measures on the attended path — there, a decision recorded what the world was; here, nothing did.

Bound: one machine, one foreign writer, a scripted model, a single mutating effect per mission, and runs measured in fractions of a second rather than hours.

A real model in the mission loop. The arms above use scripted proposers. A later suite puts a real model — reached through a command-line agent’s own login — in the proposer’s seat, on seven small mission tasks each with a checker that reads the workspace and imports nothing from the runtime: create a file, edit one key of a configuration, fix a repeated typo, rename, a task already done, the edit with a foreign write after the first look, and two dependent files. One run per task. **At the first measured commit the checker passed three of six and the runtime declared completion on one.** The binding constraint was read from the journals rather than the stop reasons: **the proposer was blind** — the runtime performed the reads the model asked for and none of their content reached the next prompt, so the model asked again until a repeat guard stopped it. Showing the proposer what the runtime observed and what became of its actions moved the suite to five of seven; three further decisions — a read can answer a checkable criterion, a criterion observed unmet is work, and arguments can reach a capability the sentence missed but never a more harmful one — moved it to **seven of seven on the checker and six of seven declared complete by the runtime**. **Across all twenty rows of the three commits, no row the runtime declared complete was failed by the checker;** the one remaining gap runs the other way, a rename that landed and could not be de-

clared. On the same seven tasks, with the same seeds, objective text and checker, Claude Code passed seven and Codex six. **The suite does not separate the three runtimes**, and the one differing cell is not a stale write: Codex read the file, saw the foreign change after its first edit merged around it, and set the value back to what it had first read — an inference from the reads on either side of that edit, since its event stream records no diff — under an objective, *leave every other setting exactly as it is*, that does not say *as it is* when. That cell is ambiguous by the task’s own wording.

What this suite does not measure, and it is the property this paper is about. No run in it asked a person anything: every mission’s authority was delegated, and the approval gate never escalated. The suite is evidence about the runtime a mechanism sits in — that a real model can drive it to completions an independent checker confirms, and that the runtime’s claim of completion was never stronger than the check — and **it is no evidence that the mechanism holds with a real model**, because the mechanism was never reached. *Bound:* one model, one run per task, synthetic seeds, seconds to minutes per task.

The cell that should reach the mechanism, and why it did not. One task was then built to reach it: a mission granted reads only must ask before rewriting a configuration file whole, a person approves once, and the world moves either after the answer or on the next run’s first look, with an unrelated-file control, an unchanged-world control, and the re-judge-removed column. The reading was fixed before the run, and every row was valid — the first run asked, the approved effect was the whole-file write, the approval’s binding carried an identity. **No row reached revalidation.** After the answer the next run re-proposed the write, and in five of seven rows its arguments differed from the approved ones, so instance binding asked the person again before any world comparison could happen — including in both unchanged-world rows under the mechanism, **an unnecessary second question with nothing moved.** The content was byte-identical in three of the four mechanism rows; the difference was optional arguments the model added or dropped between proposals, one of them the schema’s default and one of them not, so normalising to defaults would have repaired none of those four. **A proposer that regenerates what a person approved turns exact occurrence identity into a re-ask on every approval**, which is a cost of §5.5’s repair under a real model and is reported in RQ5.

Two rows are still worth reading. In the re-judge-removed column, where the model’s re-proposal happened to match, a foreign write landing on the run’s own read of the file was dispatched and **refused by the provider’s basis check alone** — the execution layer catching a stale write with no mechanism. The corresponding row under the mechanism cannot separate the layers by design: the injection landed on the model’s own read, so the re-judge had no window of its own.

Table 6. The approval task with a real model: who refused a stale write. One run per cell.

cell	with the re-judge	re-judge remove
world moves after the answer	gate: changed since approval	stale write exec
world moves on the next look	provider: stale basis	provider: stale b
nothing moves	executed, no second question	executed
unrelated file moves	executed, no second question	—

The next change, made by the runtime track, re-submits the approved proposal itself rather than asking the model to write it again; that keeps occurrence identity exact and makes the world-moves-after-the-answer row the clean mechanism cell it was meant to be.

With the approved proposal carried and the approved write grounded, the layers separate with a real model.

Two changes preceded the re-run: the next run offers the proposal the person answered, read back from the run that asked, and an approved action is grounded on the look that covered it (§5.5’s identity stays exact; RQ5 reports the missing basis this repaired). The seven rows were re-run against predictions committed beforehand, and all seven predictions held, with every row valid, every re-proposal equal to the approved one, and every dispatched basis naming a look of the run that dispatched.

A change landing between the decision and the next run’s first look was refused by the gate under the mechanism and written over without it (Table 6). In the ablation row the dispatch was grounded on a look whose file digest equals the changed file’s, not the seed’s — recomputed here from the task’s seed and the injected edit — so the provider found nothing moved, and the approved content restored the old port over the other writer’s change. **A change landing after the run’s last look was refused by the provider in both columns**, each on the look its column takes: the re-judge’s re-read under the mechanism, the pre-change look without it. The person was asked again after both provider refusals, because the model then proposed something new; that second question is recorded and counted as evidence for neither layer. **So the decision-layer and execution-layer guards each catch exactly the window the other cannot see**, on a real runtime with a real model: the witness recorded at the decision sees what changed before any look, and the basis taken at the last look sees what changed after it. *Bound:* one model, one task, one run per cell — seven traced runs, not rates; the mechanism-column refusal of the late change depends on the grounding repair, without which that cell could not have been refused by anything.

How far the binding reaches. As the runtime stood when this was measured, of the five mutating capabilities that escalate, three could carry a revalidatable approval, and only

when the target already existed; a later repair moved that bound, so the table of what binds is a statement about a version. The cases that cannot bind share no reason — the resource does not exist yet, it was observed but carries no identity, or nothing asked about it — and the defect is at a seam: the prompt asks both that a request *mention* the resources in doubt and that it *cover* them, while the check enforces only the first, so **a port that obeys the enforced half binds worse than one that ignores it.**

6 Related work

The closest work. Commit-time authorization defines a correctness condition and measures violations from outside the runtime, trusting the signals it emits; its taxonomy of observability failure is binary — a signal is emitted or missing [38]. Every failure we measure is a third state: emitted, well formed, and not load-bearing. **Its witness is the nearest published object to ours**, and what separates them is *when* the witness is taken: at the durability boundary there, at the human decision there is none. The failure is externally named as a root cause — authorization checked once and trusted forever — in a systematization of 39 execution-security papers, which also leaves our second question open: whether a specific authorized action should happen now [8]. A dedicated study of time-of-check-to-time-of-use in agents treats the failure as a window and narrows it [33]; here the window is structurally zero and the check still fails.

Stale authorization as a concurrency-control problem. A parallel line treats this as concurrency control rather than security, and it is where our nearest neighbours are. **Stateful governance names stale approval directly**, as the long-running human-review form of acting on an allow decision after the state that justified it has changed, and gives a serializability condition over policy state read through declared scopes [14]. **We claim no priority over the problem statement.** What differs is the state: a policy-state transition is declared and owned, whereas a second writer changing a file’s bytes declares nothing, and that paper’s guarantee is voided by scope under-declaration — which is the case we measure. Dependency-scoped validation checks a plan’s lineage frontier against owner-authorised heads before a protected action and blocks when it cannot decide, reporting zero invalid actions where a freshness-only executor acts on an obsolete plan in every task [12]; its records have owners and emit heads, and ours has neither, so on our mutation it would return *inconclusive* — safe, and not the stand-and-fall partition. Semantic transactions record anchors on a transaction and stage external effects, but do not version those anchors against the world, so a transaction can commit on a stale premise [11]: the record-without-revalidation case, and the reason recording a witness is not by itself the mechanism.

An ACID-by-analogy treatment of agent transactions carries no staleness detection at all [6].

An alternative we did not exclude. Where we re-check before reuse, a concurrency runtime for multi-agent systems pushes a *notification* to any in-flight reader whose premise a lower-ranked write touched, and the agent repairs [10]. This is writer-initiated invalidation, and it protects the same property by paying at write time rather than at reuse time. **We ran no invalidation comparator, and state the two reasons it does not obviously transfer as arguments, not results:** the writers it notifies are ranked participants the runtime schedules, whereas ours is behind the runtime; and a durable approval has no in-flight reader to notify between the answer and the reuse. Its falsifier is cheap — an invalidation-triggered design reaching our stand-and-fall partition on the same scenarios. Relatedly, read sets reconstructed by a runtime beat read sets agents declare, with self-reports overclaiming by 32–49% [13], which is the published warrant for taking the look through the runtime rather than trusting a declaration.

Binding an approval, and bounding authority. Consent integrity names the divergence between an approved and an executed action under an adversary who forges the narration [40]; a canonical action identity formalises the matching fix [9]. A framework audit makes the adjacent point that capability gating is not per-call authorization [34] — a question of *scope*, where the gate never asked about the argument, rather than of a gate that asked a question its representation could not form. Revocable capabilities bind authority to an episode’s lifetime [37], the second primitive of §3.

Markers that nothing consults. Revocation enforcement in agent memory has been measured across five systems, nine scenarios and nine models, and no system enforces it by default [39]; validity has been made an explicit, revocable state of memory and evaluated [42]. Our second recurring shape — a plane that exists, is computed, and nothing reads — is therefore not ours to name. Memory granting authority the underlying history never permitted [19] is the opposite of our case, where the approval is genuine, correct under its own recorded contract, and outlives the world that justified it.

Systems ancestry. A premise with a basis is optimistic concurrency control [30]: act against a snapshot and validate before committing — but a database validator sees every transaction in its own log, while ours sees one writer. The classical detectors for one resource with two writers are version vectors [1] and invalidation protocols [18]; a scalar counter is neither, and is a degenerate case of the ordering

primitive of logical clocks [32]. Linearizability [26] is a condition within a single history, whereas here two providers over one resource do not form one history at all. Isolation-level analysis [17] supplies a method we lack — name the anomaly, then say which level excludes it — although on the cross-provider path no level is consulted. Provenance semirings [25] are the formal ancestry for reconstructing where a fact came from rather than carrying it on the fact.

Choosing a trigger. Where version comparison cannot fire, the trigger classes in use are lease expiry [24], writer-initiated invalidation [27], and read-triggered repair with background anti-entropy [20]. Choosing the interval was formalised in real-time databases, where each datum carries an absolute validity interval and update periods are derived from it [41], with triggered rather than periodic updates [2] and staleness measured under real workloads [15]; the security literature measures the same window from the other side, and widens it deliberately [3]. What does not transfer is the cost assumption: each treats a check as a message, where here it is an observation charged to a budget. Deciding which checks to run [35] and bounding the overhead of production monitors [4] have their own literatures.

Shipping runtimes, read rather than measured. We read the approval and validation paths of five widely used agent runtimes. None records, on an approval, any observation of the world that could later be re-checked; two compute a digest at approval time, and both cover the *call* rather than the world — one hashes the tool arguments, with a source comment saying it does so to tell similar calls apart. **This establishes what those systems represent, not how they behave,** and we draw no behavioural conclusion from it. Two findings cut against us: rich approval vocabularies are common — one runtime’s human response is four-valued, another’s permission five-valued, and a third keeps granted and refused as separate fields — and one folds its five-valued permission into a single boolean in the durable record, the collapse we report here. Neither the distinction nor the fold is peculiar to this runtime.

Checks that pass for the wrong reason. In model checking, a property satisfied for a reason unrelated to the engineer’s intent is *vacuous* [31]. The checks of §8.1 that reported success by construction are instances, relocated to a test suite; we report instances and cheap rules against them, not the phenomenon. The testing literature supplies weaker neighbours: coverage correlating poorly with suite effectiveness [28], mutants as a substitute for real faults [29] — the warrant for our batteries — and the oracle problem [16].

7 Limitations

One runtime, one filesystem provider pair, one machine, scripted intents, no arm driving a model; no frequency is measured, because no execution history survives to be counted. A reading of five shipping runtimes finds none recording a re-checkable world premise on an approval, but that is a claim about what they *represent*, not what they *do*; the architectural form of our result is a hypothesis. The mechanism cannot reach an approval over a directory: a digest is computed over bytes and a directory has none, and such an approval degrades to single use and then refusal — toward escalation, not permission. This limit follows from the field the key reads and is not a measured arm. Production cost is unmeasured. The repair is reported as implemented, and compared against the unrepaired runtime on a bounded set over which it does not over-invalidate — not as validated. Absence cannot be bound, because the provider reports a missing path as a failure that only a namespaced code distinguishes from an unrecognised error, so approvals over creation remain single-use. An approval over an effect that names nothing records no binding and is never revalidated: the reuse path returns before revalidation when nothing is addressable, and the single-use rule applies only to a binding that names a resource without an identity (both read from source). For this population the shipped mechanism does not satisfy the property of §3: it records neither a witness nor a typed reason. Within one run a same-command reuse dispatches (measured). Nothing stops a repeat: a run-scoped repetition guard, unrelated to authority, halts the loop only after the repeated effect has run, and a resumed run starts it empty (read from source; unmeasured). *Foreign* means outside the coordinator, not outside the process: the write is a real filesystem write nothing in the runtime recorded, made from the test’s own process. A second operating-system process, a concurrent writer, and a second provider over one root are unmeasured for the repair, and an approval whose proposing intent established no premise has no arm. For a file, a missing identity is a configuration — a listing hashes its entries only when asked; for a directory it is structural, and stable under nesting. Harness arms used a look port that covers its targets; on the daemon the look is the model’s, and a nested target it does not name executes stale (§5). And an approval covers capability and resources, not content: §5 measures a re-proposal the person was never shown running on it.

Bounds added by the measurements of §5, listed rather than left in place. The nesting bound is closed in no column. Enabling the runtime-supplied precondition did not close it: the basis exists, the provider reports it as not covering the nested path, and the mode that had to be chosen passes exactly that verdict — the three-valued answer collapsing one layer below the one this paper is about, in the

permissive direction. **Three mutating capabilities remain unenforced** — delete, mkdir and move — now a named gap rather than an unexamined one, since each destroys something different and inherits no answer from the whole-file case. **The parasitism result belonged to a version of this implementation and no longer holds**: the runtime now observes before every change for its own reasons, which is the counter-example we had named, and the separation that replaces it — a witness recorded at the decision against one taken at dispatch — is measured on one cell. **That pre-change look grounds the action, not the proposal**: on the unattended path a change landing before the look is overwritten, measured, and nothing covers that window where no decision was recorded. **A covering look that is refused or fails does not block the effect**: a delegation that permits writes and denies reads dispatches its write without a basis, a named gap that the only arm exercising it misses because its scripted compiler cannot reground. **Creating a file spends a look that cannot cover it**, since a path that does not exist has no identity. And the pre-change look **does not close the nesting bound**: on the nested-target arm it fires, fails to cover the nested path, and the write executes stale; the named fix is a look the runtime composes itself, which has not been built. **On one effect shape the mechanism is worse than the provider it duplicates**: where a patch’s match still resolves and the file’s digest has moved, it raises an unnecessary rejection and the column without it executes correctly. **A resource the effect does not name is never bound and never compared**, so an approval whose justification rested on a fact about some other resource is outside the mechanism by construction. **Resume is blocked on the protocol, twice over and independently**: a thread is owned by the connection that started it, and a thread’s identifier is derived from the session’s open time, so a restarted daemon cannot name the old thread — the answer can act, and the question cannot reach the person again. **Approved writes went out without a basis until a repair**: under the mechanism the re-judge’s covering look satisfied the pre-change look’s coverage test without grounding the action, measured in every approved top-level dispatch at two commits; after the repair every dispatch carries a basis and no grid outcome moved, but the grid never injects a change after the re-judge’s look, so the repaired guard is exercised only by the real-model cell. **An approved write can act when the proposer cannot**: the carried approved proposal needs no model call, while choosing a look’s parameters does. In one attempt whose model transport had lost its login, the approved patch applied, but the criterion could not be evaluated — the observation available covered the file’s state without its content, and every model call that could have asked for more failed — so the mission stalled with the work done and unverified. The runtime track recorded it as an invalid attempt, not a measurement; its transcript alone does not show which look the re-judge compared, and we state it only as a design bound that a runtime-

composed look would remove. **Revalidation with a real model rests on one task and one run per cell**: the seven-task suite requested no approval; the approval task first never reached revalidation because a regenerated re-proposal was asked again, and reached it only after two runtime changes, as seven traced runs rather than rates. **Two costs it surfaced are standing**: routing depends on the proposer’s wording — the same arguments reached a directory-creation capability under one sentence and the write under another — and a read whose expectation holds can now evidence a criterion that carries no assertion, so a weak expectation such as *the path exists* completes an unchecked criterion, the same weakness such criteria already had for writes. **The comparator column against whole runtimes cannot speak to reuse at all**: its hook answers every call, so no approval outlives the question it answered, and one comparator’s model was not captured.

8 Discussion

Local correctness does not compose into a valid dispatch.

Figure 1 is three gates each returning a true answer and one dispatch that should not have happened. The bounded reading the measurements support is that **local correctness of each gate does not imply validity of the composed dispatch when the gates share no representation of the facts authorisation depends on**. Here they do not: the precondition gate holds session and revision, the staleness guard a provider-local counter, and the approval capability, resources and a timestamp — three vocabularies with no term in common that a foreign write moves. Each gate answers its own question completely, and none holds a representation in which the question of §2 can be posed.

Neither cause rests on one experiment. One cause is measured in four independent experiments and the other in eight, and each spans a single run, a run boundary, and a persistence-and-resume cycle. These are counts of experiments, not arms. We note it because much else here is measured once, and a result seen four times across three boundaries differs from one seen once in a way a verdict does not show.

Two recurring shapes, kept apart. *Absence read as a value* — *could not tell* recorded as *nothing moved* — recurs in this system five times, each somewhere new: a provider verdict returning a fact with no identity as *the world did not move*; a runtime fact that would have recorded *absent* as observed; an authority key under which two empty resource lists compare equal; the loop written to repair that, whose first version reported a failed look as *nobody asked*; and a criterion evaluator folding *could not be evaluated* into

not satisfied. A second shape is distinct: *declared and unconsumed*, a plane computed and read by nothing, in progressively worse forms: seams declared without consumers; two surviving mutations describing a mechanism that could have been declared, wired, and never produce a value; a flag named for a quantity it does not compute; the provenance fields of the third record of §3, present and correct and read by nothing on the path that reuses them; and, separately, the provenance link of §5 — produced, persisted, loaded back, and read by nothing, in the repair this paper proposes rather than in the runtime it examines. **We report these as observations about this system, not laws**, and keep them apart because merging them would inflate both counts and leave neither falsifiable. A third shape fits neither: the detector of §2, correct and consumed, whose output a filter drops.

What validity appears to depend on. Read as interpretation rather than as a model, an effect’s validity appears to need four things together: the decision’s reasoning was correct when made; the evidence bearing on it was observable then; the authority record stayed bound to that evidence; and a later execution can determine whether the binding still rules. The measurements separate the third from the fourth: the third record of §3 *stays* bound to its evidence, provenance intact, and fails only the fourth — nothing on the path that reuses it attempts the determination. That is the cleanest case we have that the last clause is not implied by the one before it. We offer no theorem, and have not shown the list complete or minimal.

And the list is missing something the measurements later supplied, twice. The fourth clause presumes there is something to determine *with*. First the measurements showed there need not be: an action carried a witness only when the run about to dispatch it had observed, and at that time the only thing that made that run observe was the mechanism the clause describes. Then the runtime began observing before every change for its own reasons, and the gap closed — and a sharper one was left (§5.3). A witness now exists at dispatch, but **it is a witness of the world at dispatch, not of the world the decision was about**. So the missing condition is not merely *a comparable observation exists at the moment of reuse*; it is **a witness of the state the decision was about exists at the moment of reuse**, and that witness can only be taken when the decision is made. No layer below can supply it for itself, because nothing below the decision knows when the decision happened.

8.1 Measurement discipline

The methodology findings of this work are not contributions and are not enumerated. They collapse to one principle — **a measurement must be capable of falsifying the claim it is used to support** — and three instances earn their place because each fails it in a different way. The remainder, in-

cluding the convention that refuted predictions are retained verbatim (§5.1) and a family of sentences that outlived the object they were true of, are in the artefact notes.

A check that exists but cannot fail. Two comparators central to §5 read cells out of a keyed collection and tested the two sides for inequality, so a pair of missing lookups compared nothing to nothing and reported **agreement on a comparison that never happened**. For an experiment whose finding is a null, an empty loop returns exactly the shape a null result has. A checker for the ledger’s commit column reported success four ways, once by resolving a file that was always there. Both now fail loudly, naming the missing cell, and the commit checker requires the recorded commit to have touched the experiment’s files. The harder case is a guard gated on the same precondition as the thing it protects: **correct scoping and vacuity are indistinguishable from inside the guard**, and what separates them is only whether the guard can fail while its subject is at risk — so such an arrangement’s protection is contingent rather than absent, which is weaker than our first statement of the rule and is the form that survived checking. The codebase had already named this family; a checker in the tree documents at length that it would pass by resolving nothing, and further instances shipped anyway. **Naming a failure family does not immunise against it**. The rule that follows is cheap: before trusting a green comparator, assert what it returns on empty and on truncated input. **The runtime supplied a later instance of the same shape**: two routing tests asserted that a goal completed, and the search each goal existed to run had been refused and never executed — the verification plane had credited the refusal with a world it did not make (§5.7). *Completed* read identically before the defect was fixed and after the search was made to run, and only the evidence between the two told them apart.

An instrument that was itself wrong. A record of checks is a biased sample of checking, because a check that finds nothing rarely earns an entry — the same shape as counting only the guards that fired. To estimate that bias in our own research record, a keyword classifier partitioned 38 headed sections and reported roughly two findings per null. **The classifier could not distinguish this was correct from correcting my own error**, and six of the headings it filed as null results contain the word *correction*. Hand classification of the ten it called null found eight to be corrections and therefore findings, one confirmatory and one mixed: on the order of one or two genuine nulls in 38 sections rather than ten, so the bias is considerably worse than the instrument reported, and the error ran in the direction that flattered the record. The same shape had already cost a count: for most of the study a single ledger field recorded both *predicted and wrong* and *predicted by no one*, and every count generated from it reported twice the true number of wrong predictions, again in

the flattering direction, since *unforeseen* was being counted as *caught*. Two fields replaced it. **Instrument validation cannot be assumed because an instrument is simple, or because it was built locally for one question.** A known-answer case would have caught either in one run.

A plausible number that suppressed its own re-measurement. This paper’s arm total stood at 269 after the ledger had reached 272, because 269 was once true and still looked right. **The generator that would have contradicted it existed and had not been re-run**, and the experiment its table omitted was the instance-binding repair of §5.5 — the strongest result in the evaluation. Two derived counts in the same paragraph, of refuted and of unpredicted arms, had been re-derived and were correct, which is what let the stale total pass inspection beside them. A number that looks right occupies the slot a measurement would have filled, and the defence is re-derivation on a schedule rather than vigilance. The setup table is generated from the ledger and the ledger’s own totals are re-derived rather than incremented; **the counts inside prose are not generated**, and that remains the exposed surface.

9 Reproducibility

Every result here is a test in the repository, and every arm names the commit it resolves in. A check in the tree verifies that each recorded commit touched its experiment’s files, so an arm cannot cite a commit that did not produce it. The setup table and the result figure are *generated* from the ledger rather than maintained beside it — twice during this work a committed artefact was stale while its generator was correct, and both times the discrepancy was found by re-running the generator rather than by reading it, which is why the paper says which artefacts are generated and treats the claim as checkable.

Evidence verifies from a clone, and that is a stronger statement than a manifest existing. Nine evidence slices carry thirty-three SHA-256 manifests over 1,288 files, and **all 1,288 are files git actually carries** — checked by intersecting every manifest entry with `git ls-files`, because a manifest that hashes files the repository does not track verifies only in the tree that wrote it. That was once true of six directories at once here, while the text cited those manifests as its evidence. Every manifest was also re-verified against the files it names, and none failed. The real-model mission runs of §5.7 were committed for a time with no manifest; each of their run directories now carries one.

One naming convention, stated because it looks like a missing artefact. An evidence directory is named for the commit the run was measured *against*, not the commit that carries it: committing a record necessarily makes a new commit, so `evidence/<slice>/<anchor>/` can never exist at

`<anchor>` itself. Checking out an anchor and finding nothing at the cited path is the convention working.

Ablation columns are reproducible as a procedure, not only as a result. A baseline is the same tree with one mechanism removed at *exactly one site*, never a historical checkout, and an edit that fails to land at exactly one site is refused — because an edit that does not land makes every cell agree and reads exactly like an inert repair. Each column’s dump is produced by one command against the shipped binary, and the dump records the diff it applied, so a reader can confirm the ablation was the single site claimed. **This paper’s authors reproduced the decisive ablation independently of the track that ran it**, and the readings agreed.

The external comparators are pinned at the supply chain, not at a version string. Wheels were hashed, then installed from the hashed files with the index disabled, so what was hashed is what ran; 409 installed files were checked against the hashes their own wheels declare; and the recorded digests were compared against what the registry published for those exact pins, from a snapshot committed before the comparison.

What does not reproduce, stated rather than left for a reader to discover. The whole-runtime comparator column drives real models, so it is not deterministic, and one of the two runtimes did not record which model it used — a provenance gap, not a detail. No arm anywhere drives a model in the runtime under test, so nothing here reproduces a latency. And the papers build with a portable $\text{\TeX}$ toolchain because no system installation exists on the machine that wrote them, which is worth knowing before a build is attempted.

10 Conclusion

A human approval is a memoized decision, and an agent runtime can check it at every gate while holding no key against which the answer could fail. Recording the state a decision was about, and keeping *no witness* distinct from *witness differs*, is a property a runtime can be checked against. Validity is one of two orthogonal primitives; entitlement to act is the other.

References

- [1] Comments on “detection of mutual inconsistency in distributed systems”. *IEEE Transactions on Software Engineering*, 1987. Cited for the version-vector construction of Parker et al., 1983.
- [2] Triggered updates for temporal consistency in real-time databases. *Real-Time Systems*, 2000.
- [3] Exploiting Unix file-system races via algorithmic complexity attacks. In *Proc. 30th IEEE Symposium on Security and Privacy*, 2009.

- [4] Adaptive runtime verification. In *Runtime Verification*, LNCS, 2013.
- [5] ACLE-MCP: Attested capability leases for execution-time trust in remote LLM tool use, 2026. arXiv:2609.02690v1.
- [6] Agentic transaction: Towards ACID-compliant agent systems, 2026. arXiv:2608.13900v1.
- [7] Atomicity for agents: Exposing, exploiting, and mitigating TOCTOU vulnerabilities in browser-use agents, 2026. arXiv:2603.00476v1.
- [8] The balkanization of execution-security research for AI coding agents: Isolation, access control, and time-of-check-to-time-of-use vulnerabilities, 2026. arXiv:2607.05743v1.
- [9] CAVA: Canonical action verification and attestation for runtime governance of agentic AI systems, 2026. arXiv:2607.13716v1.
- [10] CoAgent: Concurrency control for multi-agent systems, 2026. arXiv:2606.15376v1.
- [11] Cordon: Semantic transactions for tool-using LLM agents, 2026. arXiv:2606.17573v1.
- [12] Fresh memory, stale plans: Dependency-scoped validation for distributed LLM-agent memory, 2026. arXiv:2609.03340v1.
- [13] S-Bus: Automatic read-set reconstruction for multi-agent LLM state coordination, 2026. arXiv:2605.17076v2.
- [14] Stateful governance for concurrent agentic systems, 2026. arXiv:2608.02764v2.
- [15] Peter Bailis, Shivaram Venkataraman, Michael J. Franklin, Joseph M. Hellerstein, and Ion Stoica. Probabilistically bounded staleness for practical partial quorums. Technical report, 2012.
- [16] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing. In *Proc. 26th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2017.
- [17] Hal Berenson, Phil Bernstein, Jim Gray, Jim Melton, Elizabeth O’Neil, and Patrick O’Neil. A critique of ANSI SQL isolation levels. *ACM SIGMOD Record*, 1995.
- [18] Lucien M. Censier and Paul Feautrier. A new solution to coherence problems in multicache systems. *IEEE Transactions on Computers*, 1978.
- [19] Tommaso Cerruti, Mika Okamoto, and Ansel Kaplan Erol. Agent memory is a surface for endogenous authorization laundering, 2026. arXiv:2609.01836v1.
- [20] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Voshall, and Werner Vogels. Dynamo: Amazon’s highly available key-value store. In *Proc. 21st ACM SIGOPS Symposium on Operating Systems Principles*, 2007.
- [21] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.
- [22] R. S. Fabry. Capability-based addressing. *Communications of the ACM*, 17(7):403–412, 1974.
- [23] Virgil D. Gligor. Review and revocation of access privileges distributed through capabilities. *IEEE Transactions on Software Engineering*, SE-5(6):575–586, 1979.
- [24] Cary G. Gray and David R. Cheriton. Leases: An efficient fault-tolerant mechanism for distributed file cache consistency. In *Proc. 12th ACM Symposium on Operating Systems Principles*, 1989.
- [25] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. In *Proc. 26th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 2007.
- [26] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM Transactions on Programming Languages and Systems*, 1990.
- [27] John H. Howard, Michael L. Kazar, Sherri G. Menees, David A. Nichols, M. Satyanarayanan, Robert N. Sidebotham, and Michael J. West. Scale and performance in a distributed file system. *ACM Transactions on Computer Systems*, 1988.
- [28] Laura Inozemtseva and Reid Holmes. Coverage is not strongly correlated with test suite effectiveness. In *Proc. 36th International Conference on Software Engineering*, 2014.
- [29] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. Are mutants a valid substitute for real faults in software testing? In *Proc. 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2014.
- [30] H. T. Kung and John T. Robinson. On optimistic methods for concurrency control. In *Fifth International Conference on Very Large Data Bases*, 1979.
- [31] Orna Kupferman and Moshe Y. Vardi. Vacuity detection in temporal model checking. *Formal Methods in System Design*, 2001.
- [32] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 1978. Collected edition.
- [33] Derek Lilienthal and Sanghyun Hong. Mind the gap: Time-of-check to time-of-use vulnerabilities in LLM-enabled agents, 2025. arXiv:2508.17155v1.
- [34] David Mellafe Zuvic. Capability gates are not authorization: Confused-deputy failures in LLM agent frameworks, 2026. arXiv:2606.28679v1.
- [35] Gregg Rothermel and Mary Jean Harrold. Analyzing regression test selection techniques. *IEEE Transactions on Software Engineering*, 1996.
- [36] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.
- [37] Igor Santos-Grueiro. Lingering authority: Revocable resource-and-effect capabilities for coding agents, 2026. arXiv:2606.22504v1.
- [38] Igor Santos-Grueiro. Temporary authority, permanent effects: Commit-time authorization for LLM agents, 2026. arXiv:2607.10487v1.

- [39] Yi Ting Shen, Kentaroh Toyoda, and Alex Leung. Revoked but still authoritative: An empirical study of revocation enforcement in agent-memory systems, 2026. arXiv:2609.08258v1.
- [40] Xiaoqi Weng. What you approve is what executes: Consent integrity for black-box LLM agents, 2026. arXiv:2606.02668v1.
- [41] Ming Xiong and Krithi Ramamritham. Deriving deadlines and periods for real-time update transactions. In *Proc. 20th IEEE Real-Time Systems Symposium*, 1999.
- [42] Yan Zhou, Yue Ouyang, Kaiyang Zheng, and Suncheng Xiang. TEPA: Revoking stale memories for conflict-robust language agents, 2026. arXiv:2608.07429v2.